



Proxy

Design Patterns

A deep architectural dive into structural class decomposition and interface control surrogates.

Instructor

Md Samsuddoha

Assistant Professor
Department of CSE
University of Barishal.

Prepared by:

**Full Stack Alchemists
Team.**

Table of Contents

01 What is Design Pattern?

02 Why Structural Pattern?

03 Proxy Pattern Overview

04 Before Proxy: Problems

05 After Proxy: Solution

06 UML Structure

07 Types of Proxy

08 Operational Workflow

09 Java Implementation

10 Real-World Analogy

11 Industry Applications

12 Advantages

13 Disadvantages

14 When to Use

15 When NOT to Use

16 References

17 Q&A

Defining Design Patterns



The Definition

Standardized, field-proven blueprints solving recurring object-oriented design challenges. They are abstract architectural structures, not boilerplate code templates.



Architectural Intent

To establish a clean separation of concerns, optimize component cohesion, reduce interface coupling, and build a unified design vocabulary for developers.



Structural Context

Unlike Creational (instantiation) or Behavioral (runtime communication), **Structural patterns** explain how to compose classes and objects into larger systems.



Architecture Note: Both the Bridge and Proxy patterns belong to the Structural category. While Bridge prevents compile-time inheritance explosion, Proxy manages lifecycle & network boundary access.

The Engineering Value



Maintainability & Scaling

Enforces clear class boundaries. Minimizes cascading system regression when modifying core logical units or changing runtime implementation targets.



High Reuse & Cohesion

Allows distinct modules to fulfill exactly one responsibility (SRP) perfectly, allowing developers to safely recombine them without structural re-engineering.



Decoupling Abstractions

Enforces the Dependency Inversion Principle. Highly decoupled elements allow parallel module development and simplified mock-based unit testing.



Extensibility Protection

Aligns codebases with the Open/Closed Principle. Developers can inject entirely new runtime logic branches without altering client interface contracts.

Why is **Proxy** called **Structural**?

Focuses on structure of classes and objects

Concerned with how classes and objects are composed to form larger structures

Defines relationships between components

Identifies clear relationships between classes, objects, and interfaces

Helps organize objects in an efficient way

Arranges components into scalable, maintainable systems

Improves flexibility and reusability of the system

Enables reuse of existing structures without altering their source code

Other Examples

Adapter

Connects incompatible interfaces so they can work together

Bridge

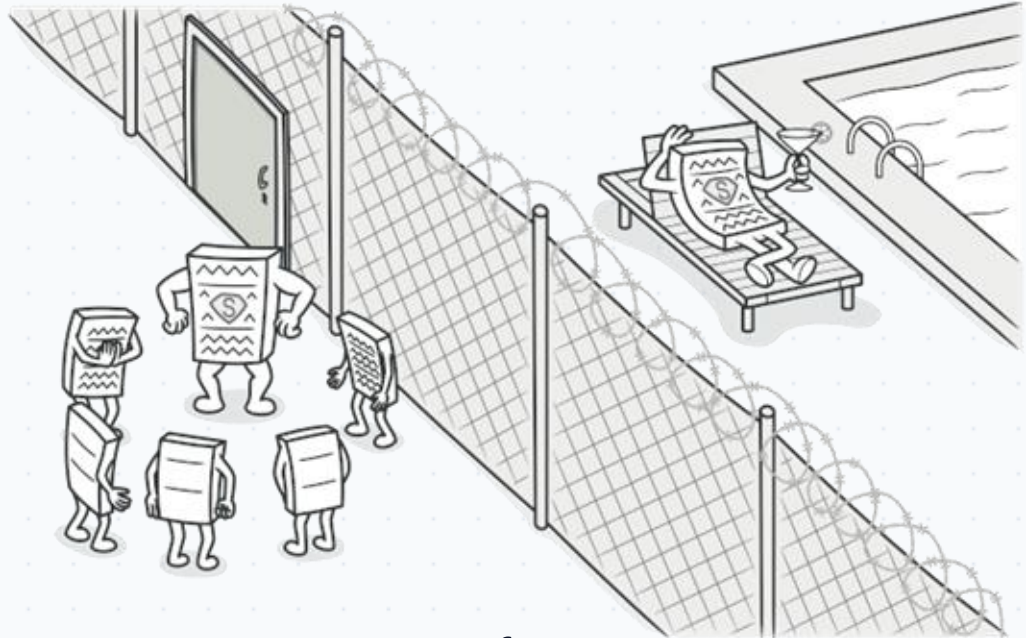
Separates abstraction from implementation so both can evolve independently

Facade

Provides a simplified interface to a complex subsystem

Official Intent

//



"Provide a surrogate or placeholder for another object to control access to it."

— Gang of Four (GoF)

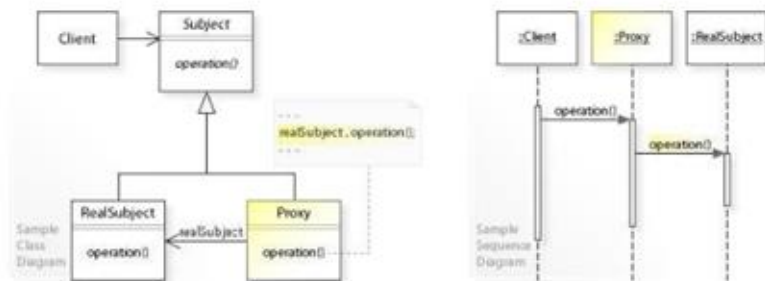
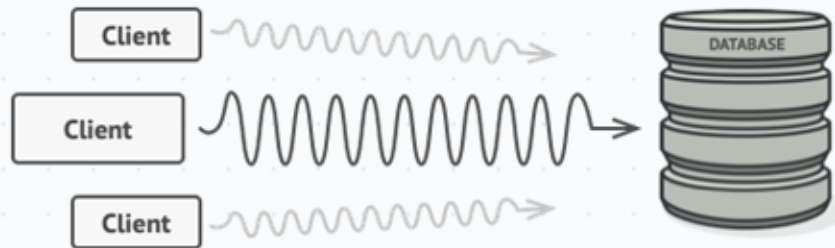
Problem Statement

Uncontrolled Access

Sometimes you need to control access to an object.

Examples include:

- ✓ **Resource Constraints:** Creating a massive object takes significant time/memory (Virtual Proxy).
- ✓ **Security:** Clients should only access specific methods based on permissions (Protection Proxy).
- ✓ **Distribution:** The actual object exists in a different memory space or server (Remote Proxy).





Before Proxy — Direct Access Problems

Architecture Without Proxy (Client directly calls every service)



The Proxy Solution



The Surrogate

A Proxy object has the same interface as the Real Subject. It acts as the "Middleman" for the client.



Access Control

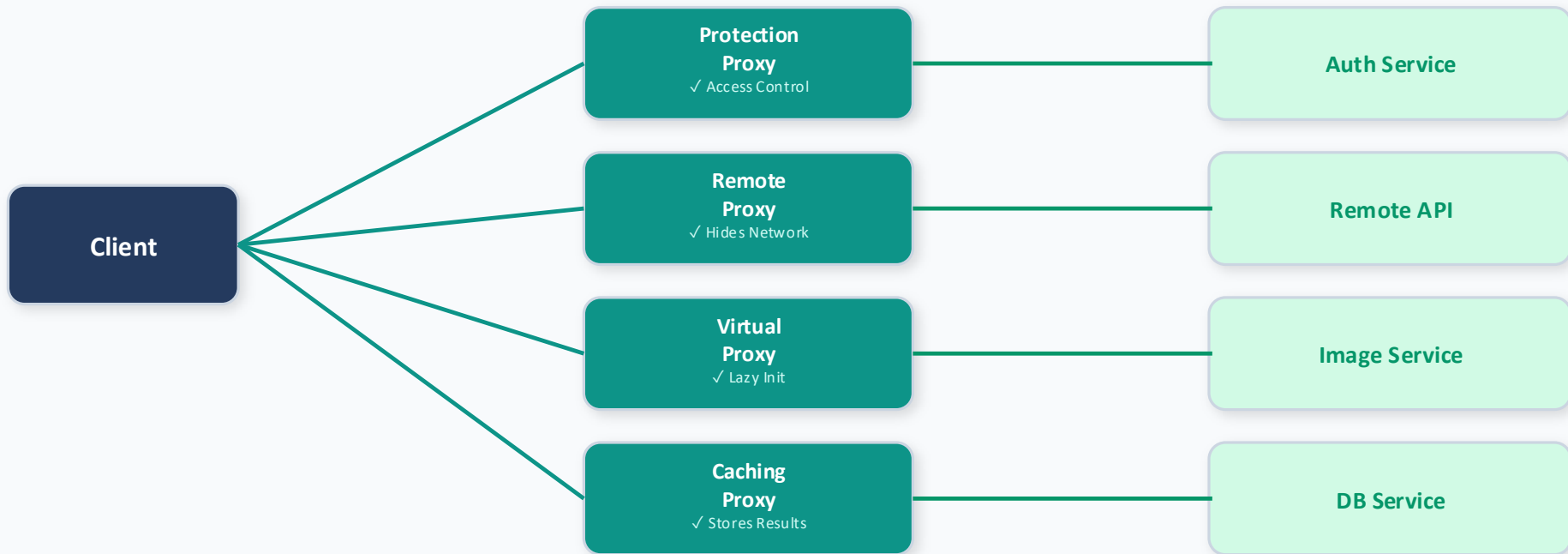
The Proxy manages the lifecycle of the Real Subject—creating it only when needed or verifying authorization before delegating calls.

Client → Proxy → [Access Check/Lazy Load] → Real Subject

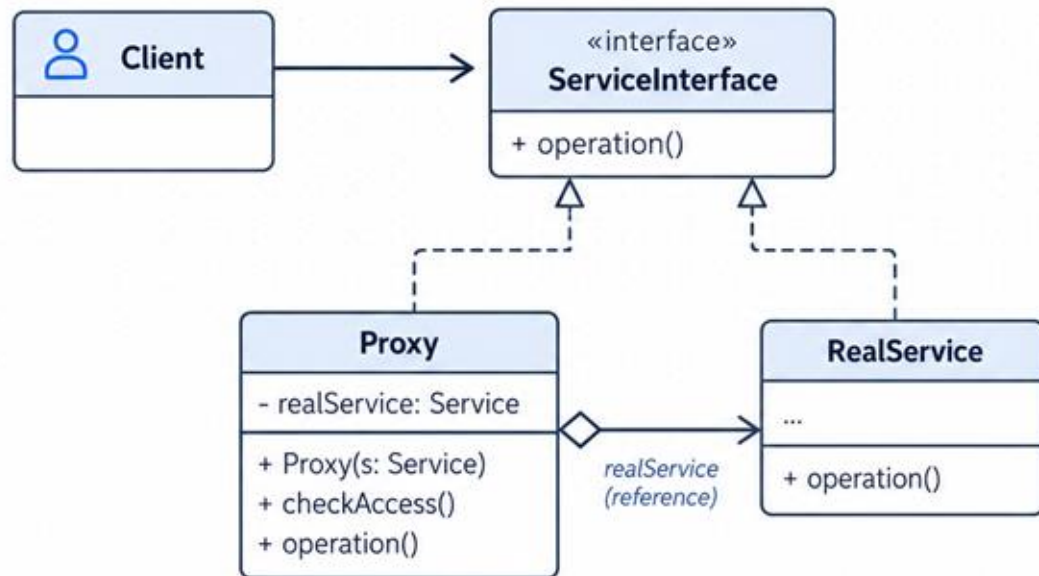


After Proxy — Clean, Controlled Access

Architecture With Proxy — Client → Proxy → Real Service (never direct)



Proxy Pattern – UML Structure



Key idea: The client can use the proxy just like the real service because both share the same interface.

1 Service Interface: Declares the common interface used by both Proxy and RealService.

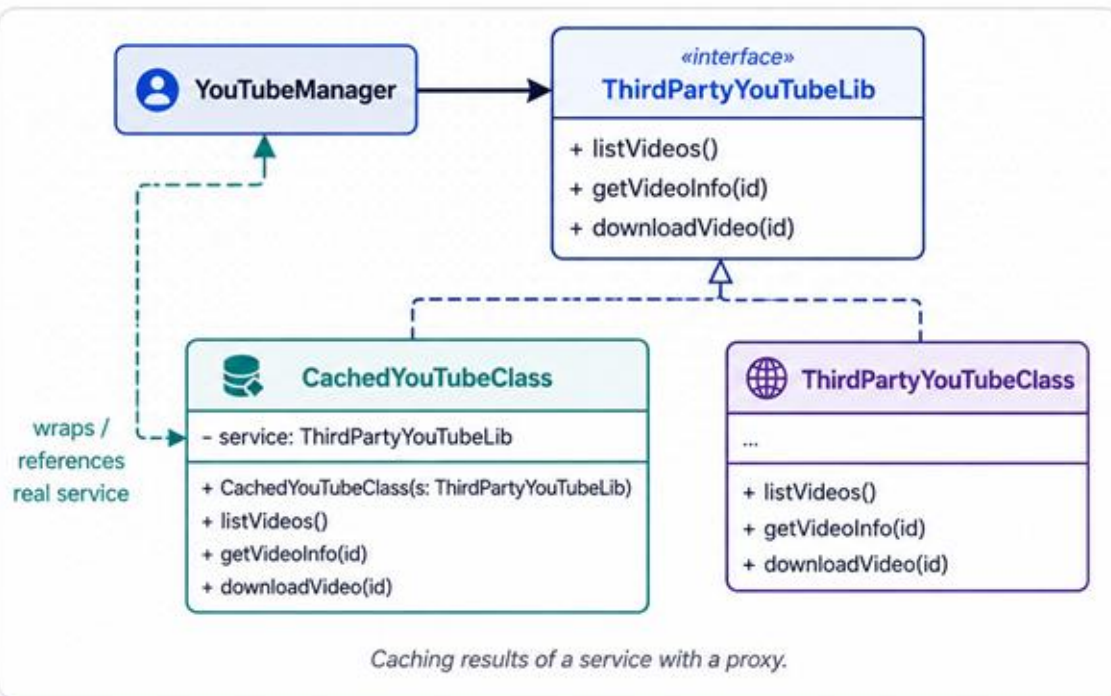
2 RealService: Contains the actual business logic and performs the real operation.

3 Proxy: Controls access and may add logging, caching, security, or lazy loading before forwarding the request.

4 Client: Works with both Proxy and RealService through the same interface.

Proxy Pattern Example: YouTube Caching Proxy

Using lazy initialization and caching in a 3rd-party YouTube integration library



1 Problem



Repeated requests cause the library to download or fetch the same data again and again, wasting time and bandwidth.

2 Proxy Solution



`CachedYouTubeClass` implements the same interface, forwards real requests when needed, and returns cached data for repeated requests.

3 Benefits



Faster response, reduced network calls, lower bandwidth usage, and cleaner client code.



Key idea: The client keeps using the same interface, while the proxy adds caching and lazy initialization without modifying the 3rd-party library.

Main Types of Proxy Pattern



Virtual Proxy

Purpose

Delays object creation until it is needed.

Benefit

Saves memory and improves performance.

Example: Large image lazy loading.



Remote Proxy

Purpose

Represents an object on another server or address space.

Benefit

Hides network complexity.

Example: RMI, gRPC or web service.



Protection Proxy

Purpose

Controls access using roles, permission or authentication.

Benefit

Improves security.

Example: Admin-only data deletion.



Smart Proxy

Purpose

Adds extra actions while accessing an object.

Benefit

Supports logging and resource management.

Example: Reference counting or audit logs.

Summary: Different proxies solve different problems — performance, remote access, security, and resource control.

How the Proxy Pattern Works

Operational Workflow



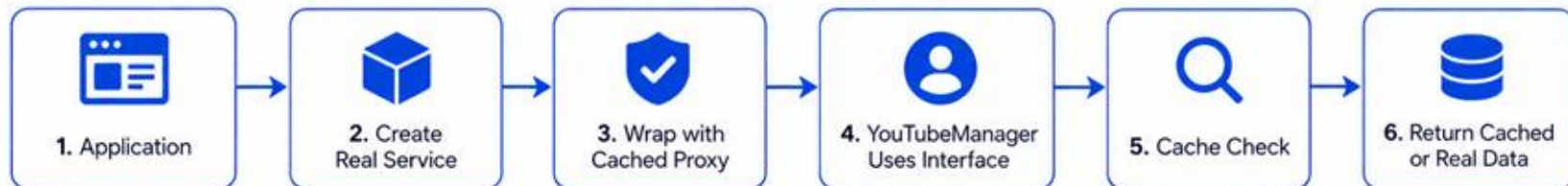
Step-by-step explanation

- 1 Client sends a request to the Proxy instead of directly calling RealService.
- 2 Proxy receives and evaluates the request.
- 3 Proxy may perform pre-processing such as authentication, validation, cache lookup, or lazy initialization.
- 4 If valid, Proxy forwards the request to RealService.
- 5 RealService executes the actual business logic.
- 6 Proxy may log, cache the result, close connection, and return the final response.

Key observation: The client remains unaware of whether the request is handled directly or indirectly through a Proxy.

How the YouTube Proxy Example Works

Request flow and class responsibilities



Class responsibilities



ThirdPartyYouTubeLib

Common interface: `listVideos()`, `getVideoInfo(id)`, `downloadVideo(id)`.



ThirdPartyYouTubeClass

Real service that talks to YouTube and fetches actual data.



CachedYouTubeClass

Proxy that stores results and avoids repeated network calls.



Step-by-step flow

- 1 Application creates `ThirdPartyYouTubeClass`.
- 2 Application wraps it with `CachedYouTubeClass`.
- 3 `YouTubeManager` receives the proxy through the interface.
- 4 When the manager requests video data, the proxy checks its cache first.
- 5 If cached data exists, it is returned immediately; otherwise the proxy calls the real service and stores the result.



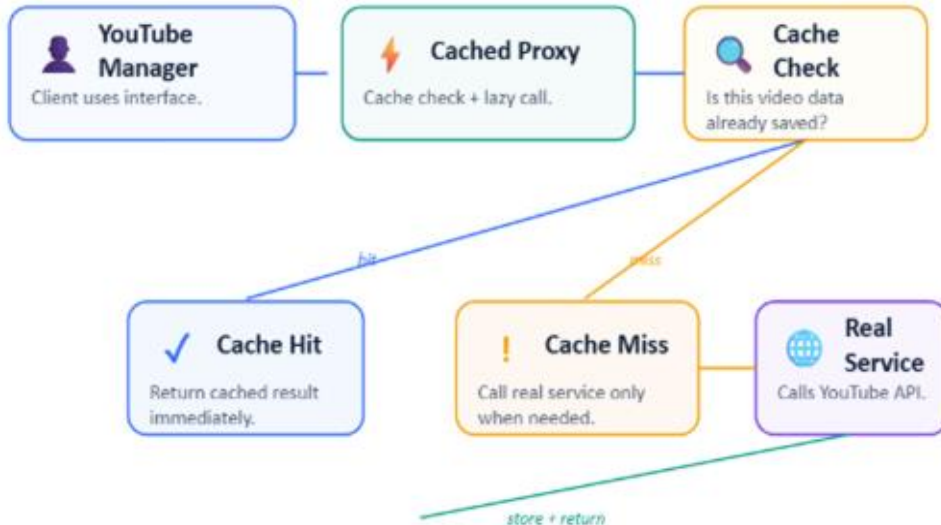
Takeaway: The GUI and manager code remain unchanged, because both the proxy and the real service implement the same interface. This makes the app faster and more efficient without modifying the 3rd-party library.

Pseudocode Flow: YouTube Caching Proxy

Lazy initialization + caching decision logic

Request flow

Proxy checks cache before calling the slow real service



Minimal pseudocode

Code-like logic that matches the diagram

```
function requestVideo(id):
    if id in cache:
        return cache[id]

    data = realService.fetch(id)
    cache[id] = data
    return data
```

What this means:

- 1) Return from cache when data exists.
- 2) Call real service only on cache miss.
- 3) Save new result for future requests.

Takeaway:

The proxy keeps the client code unchanged, but adds caching to make repeated YouTube requests faster and cheaper.

PART 1: WITHOUT PROXY

JAVA IMPLEMENTATION

Structural Overview: Without Proxy

A direct, coupled architecture where the client depends on the concrete server implementation.



Video.java – Data model



YouTubeServer.java – Expensive service

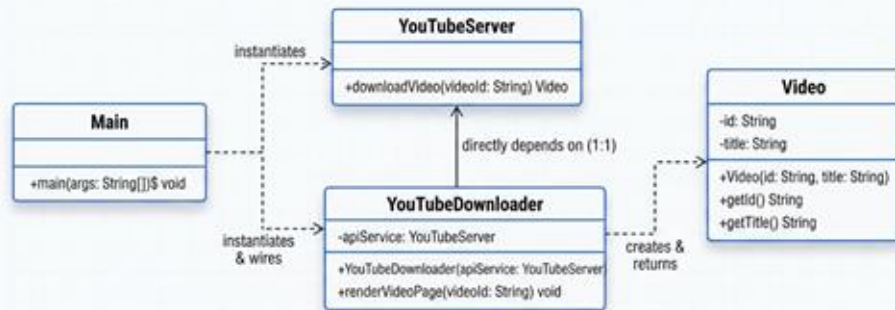


YouTubeDownloader.java – Client helper



Main.java – Entry point

WithoutProxy Project Structure



Note: This structure lacks any middle layer. Every time the downloader is used, it must instantiate and call the heavy server class directly.

Video.java — The Data Model

```
public class Video {  
    private final String id;  
    private final String title;  
  
    public Video(String id, String title) {  
        this.id = id;  
        this.title = title;  
    }  
  
    public String getId() {  
        return id;  
    }  
  
    public String getTitle() {  
        return title;  
    }  
}
```

YouTubeServer.java — Expensive Service

```
public class YouTubeServer {  
  
    // Simulates a heavy network request to download a video  
    public Video downloadVideo(String videoId) {  
        System.out.println(x: "[SERVER] Connecting to YouTube Server... ");  
        System.out.println("[SERVER] Downloading byte stream for video ID: " + videoId);  
  
        // Simulating network latency  
        try {  
            Thread.sleep(millis: 1500);  
        } catch (InterruptedException e) {  
            Thread.currentThread().interrupt();  
        }  
  
        String title = "Sample Video (" + videoId + ")";  
        System.out.println("[SERVER] Successfully fetched '" + title + "' from remote server.");  
        return new Video(videoId, title);  
    }  
}
```

- 
1. Latency Simulation
 2. Interrupt Exception Handling

YouTubeDownloader.java — Client Helper

```
public class YouTubeDownloader {  
    // Tightly coupled dependency to the concrete server implementation  
    private final YouTubeServer apiService;  
  
    public YouTubeDownloader(YouTubeServer apiService) {  
        this.apiService = apiService;  
    }  
  
    // Renders the video page by requesting it directly from the server every time  
    public void renderVideoPage(String videoId) {  
        System.out.println(x: "\n_____");  
        System.out.println("[CLIENT] User opened video page for ID: " + videoId);  
  
        Video video = apiService.downloadVideo(videoId);  
  
        System.out.println("[CLIENT] Rendering video page: " + video.getTitle());  
        System.out.println(x: "_____");  
    }  
}
```

→ Tightly Coupled Dependency

→ Constructor Injection

Main.java — Entry Point

```
public class Main {  
    Run | Debug | Run main | Debug main  
    public static void main(String[] args) {  
        System.out.println(x: "≡ Running Project 1: WITHOUT Proxy Pattern ≡");  
  
        YouTubeServer server = new YouTubeServer();  
        YouTubeDownloader downloader = new YouTubeDownloader(server);  
  
        // Requesting the same video multiple times sequentially  
        downloader.renderVideoPage(videoId: "Video 1");  
        downloader.renderVideoPage(videoId: "Video 1"); // Causes redundant server hit  
        downloader.renderVideoPage(videoId: "Video 2");  
        downloader.renderVideoPage(videoId: "Video 2"); // Causes redundant server hit  
        downloader.renderVideoPage(videoId: "Video 1"); // Causes another redundant server hit  
        downloader.renderVideoPage(videoId: "Video 3");  
    }  
}
```

Execution Output — Baseline

=== Running Project 1: WITHOUT Proxy Pattern ===

1 [CLIENT] User opened video page for ID: Video 1
[SERVER] Connecting to YouTube Server...
[SERVER] Downloading byte stream for video ID: Video 1
[SERVER] Successfully fetched 'Sample Video (Video 1)'
[CLIENT] Rendering video page: Sample Video (Video 1)

2 [CLIENT] User opened video page for ID: Video 1
[SERVER] Connecting to YouTube Server...
[SERVER] Downloading byte stream for video ID: Video 1
[SERVER] Successfully fetched 'Sample Video (Video 1)'
[CLIENT] Rendering video page: Sample Video (Video 1)

3 [CLIENT] User opened video page for ID: Video 2
[SERVER] Connecting to YouTube Server...
[SERVER] Downloading byte stream for video ID: Video 2
[SERVER] Successfully fetched 'Sample Video (Video 2)'
[CLIENT] Rendering video page: Sample Video (Video 2)

4 [CLIENT] User opened video page for ID: Video 2
[SERVER] Connecting to YouTube Server...
[SERVER] Downloading byte stream for video ID: Video 2
[SERVER] Successfully fetched 'Sample Video (Video 2)'
[CLIENT] Rendering video page: Sample Video (Video 2)

5 [CLIENT] User opened video page for ID: Video 1
[SERVER] Connecting to YouTube Server...
[SERVER] Downloading byte stream for video ID: Video 1
[SERVER] Successfully fetched 'Sample Video (Video 1)'
[CLIENT] Rendering video page: Sample Video (Video 1)

6 [CLIENT] User opened video page for ID: Video 3
[SERVER] Connecting to YouTube Server...
[SERVER] Downloading byte stream for video ID: Video 3
[SERVER] Successfully fetched 'Sample Video (Video 3)'
[CLIENT] Rendering video page: Sample Video (Video 3)

PART 2: WITH PROXY

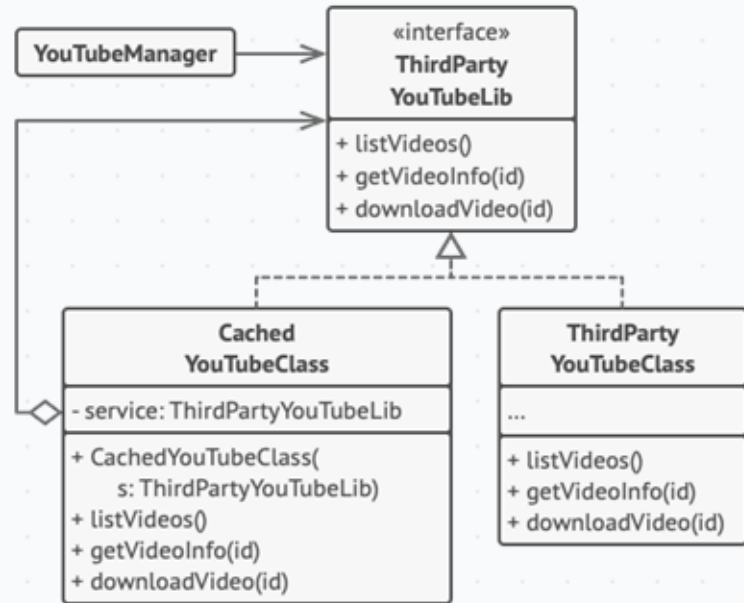
JAVA IMPLEMENTATION

The Pattern Logic

The Proxy acts as an intermediary. It provides a substitute object that controls access to the original.

Implementation Strategy:

- ❑ Define a common Interface.
- ❑ The Proxy intercepts the request.
- ❑ If data exists in Cache (HashMap), return it.
- ❑ Otherwise, delegate to the Real Subject.



The Interface Contact

```
public interface ThirdPartyYouTubeLib {  
    Video downloadVideo(String videoId);  
}
```

The Real Service

```
public class ThirdPartyYouTubeClass implements ThirdPartyYouTubeLib {  
  
    @Override  
    public Video downloadVideo(String videoId) {  
        System.out.println(x: "[SERVER] Connecting to remote YouTube Server architecture ... ");  
        System.out.println("[SERVER] Downloading heavy byte stream for video ID: " + videoId);  
  
        // Emulating heavy network overhead  
        try {  
            Thread.sleep(millis: 1500);  
        } catch (InterruptedException e) {  
            Thread.currentThread().interrupt();  
        }  
  
        String title = "Optimized Video (" + videoId + ")";  
        System.out.println("[SERVER] Successfully pulled '" + title + "' across the network backend.");  
        return new Video(videoId, title);  
    }  
}
```

The Heart : Caching Proxy

```
public class CachedYouTubeClass implements ThirdPartyYouTubeLib {  
    private final ThirdPartyYouTubeLib realService;  
    private final Map<String, Video> cache;  
  
    public CachedYouTubeClass(ThirdPartyYouTubeLib var1) {  
        this.realService = var1;  
        this.cache = new HashMap();  
    }  
  
    public Video downloadVideo(String var1) {  
        if (this.cache.containsKey(var1)) {  
            System.out.println("[PROXY] >>> CACHE HIT <<< Found metadata locally for ID: " + var1);  
            return (Video)this.cache.get(var1);  
        } else {  
            System.out.println("[PROXY] >>> CACHE MISS <<< No local record for ID: " + var1 + ". Delegating to Real Service.");  
            Video var2 = this.realService.downloadVideo(var1);  
            this.cache.put(var1, var2);  
            System.out.println("[PROXY] Persisted video object '" + var2.getTitle() + "' inside local cache storage.");  
            return var2;  
        }  
    }  
}
```

Java Collection
#Key: videoID
#Value: video

The Refactored Client

```
public class YouTubeDownloader {  
    private final ThirdPartyYouTubeLib apiService;  
  
    // Accepts the interface polymorphism wrapper  
    public YouTubeDownloader(ThirdPartyYouTubeLib apiService) {  
        this.apiService = apiService;  
    }  
  
    public void renderVideoPage(String videoId) {  
        System.out.println(x: "\n_____");  
        System.out.println("[CLIENT] User opened video page for ID: " + videoId);  
  
        // Polymorphic invocation calls Proxy or Real Subject seamlessly  
        Video video = apiService.downloadVideo(videoId);  
  
        System.out.println("[CLIENT] Rendering video page: " + video.getTitle());  
        System.out.println(x: "_____");  
    }  
}
```

Main.java

```
public class Main {  
    Run | Debug | Run main | Debug main  
    public static void main(String[] args) {  
        System.out.println(x: "=== Running Project 2: WITH Proxy Pattern ===");  
  
        // 1. Initialize the Target Real Subject dependency  
        ThirdPartyYouTubeLib realService = new ThirdPartyYouTubeClass();  
  
        // 2. Wrap the Real Subject inside our Caching Proxy interceptor layer  
        ThirdPartyYouTubeLib proxyService = new CachedYouTubeClass(realService);  
  
        // 3. Pass the Proxy into the client container application seamlessly  
        YouTubeDownloader downloader = new YouTubeDownloader(proxyService);  
  
        // Run download routines to verify cache state mutations  
        downloader.renderVideoPage(videoId: "Video 1"); // 1st time: CACHE MISS → Server Hit  
        downloader.renderVideoPage(videoId: "Video 1"); // 2nd time: CACHE HIT → Zero Server Overhead  
        downloader.renderVideoPage(videoId: "Video 2"); // 1st time: CACHE MISS → Server Hit  
        downloader.renderVideoPage(videoId: "Video 2"); // 3rd time: CACHE HIT → Instant Return  
        downloader.renderVideoPage(videoId: "Video 1"); // 4th time: CACHE HIT → Instant Return  
        downloader.renderVideoPage(videoId: "Video 3"); // 1st time: CACHE MISS  
    }  
}
```

Execution Output : Optimized

```
=== Running Project 2: WITH Proxy Pattern ===
```

```
-----  
[CLIENT] User opened video page for ID: Video 1  
[PROXY] >>> CACHE MISS <<< No local record for ID: Video 1. Delegating to Real Service.  
[SERVER] Connecting to remote YouTube Server architecture...  
[SERVER] Downloading heavy byte stream for video ID: Video 1  
[SERVER] Successfully pulled 'Optimized Video (Video 1)' across the network backend.  
[PROXY] Persisted video object 'Optimized Video (Video 1)' inside local cache storage.  
[CLIENT] Rendering video page: Optimized Video (Video 1)  
-----
```

1

```
-----  
[CLIENT] User opened video page for ID: Video 1  
[PROXY] >>> CACHE HIT <<< Found metadata locally for ID: Video 1  
[CLIENT] Rendering video page: Optimized Video (Video 1)  
-----
```

2

```
-----  
[CLIENT] User opened video page for ID: Video 2  
[PROXY] >>> CACHE MISS <<< No local record for ID: Video 2. Delegating to Real Service.  
[SERVER] Connecting to remote YouTube Server architecture...  
[SERVER] Downloading heavy byte stream for video ID: Video 2  
[SERVER] Successfully pulled 'Optimized Video (Video 2)' across the network backend.  
[PROXY] Persisted video object 'Optimized Video (Video 2)' inside local cache storage.  
[CLIENT] Rendering video page: Optimized Video (Video 2)  
-----
```

3

Execution Output : Optimized

```
-----  
[CLIENT] User opened video page for ID: Video 2  
[PROXY] >>> CACHE HIT <<< Found metadata locally for ID: Video 2  
[CLIENT] Rendering video page: Optimized Video (Video 2)  
-----
```

4

```
-----  
[CLIENT] User opened video page for ID: Video 1  
[PROXY] >>> CACHE HIT <<< Found metadata locally for ID: Video 1  
[CLIENT] Rendering video page: Optimized Video (Video 1)  
-----
```

5

```
-----  
[CLIENT] User opened video page for ID: Video 3  
[PROXY] >>> CACHE MISS <<< No local record for ID: Video 3. Delegating to Real Service.  
[SERVER] Connecting to remote YouTube Server architecture...  
[SERVER] Downloading heavy byte stream for video ID: Video 3  
[SERVER] Successfully pulled 'Optimized Video (Video 3)' across the network backend.  
[PROXY] Persisted video object 'Optimized Video (Video 3)' inside local cache storage.  
[CLIENT] Rendering video page: Optimized Video (Video 3)  
-----
```

6

Real-World Analogy

Proxy Design Pattern Explained with Real-world Example: ATM System

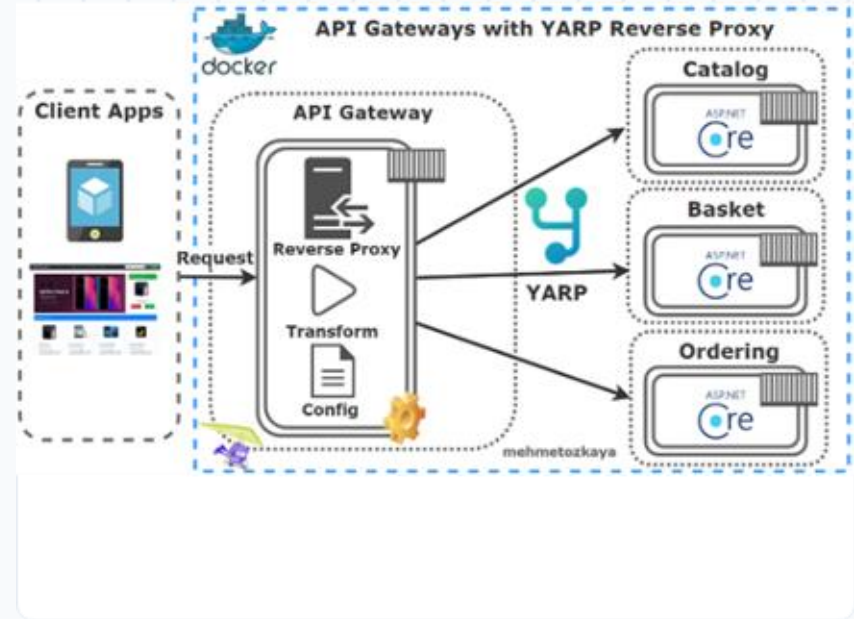


The ATM Card

- ✓ **Client:** Customer/User who wants to withdraw cash from the bank account.
- ✓ **Proxy:** Acts as an intermediary between the customer and the bank account. It verifies the PIN and checks access before forwarding the request.
- ✓ **Real Object (Bank Account) :** Contains the actual money and performs the real withdrawal operation.

Industry Applications

- ✓ **Hibernate (Lazy Loading):** Uses Virtual Proxies to avoid loading massive related objects from the DB unless explicitly accessed.
- ✓ **Spring Security:** Uses **Protection Proxy** to control access to methods and perform authorization checks (e.g., @PreAuthorize).
- ✓ **API Gateways:** Act as a Remote Proxy/Reverse Proxy for microservices, handling Rate Limiting and Logging.



Advantages



Access Control

The proxy can restrict access to the real object based on permissions — only authorized clients get through.



Lazy Initialization

Heavy objects are only created when truly needed, saving memory and speeding up startup time.



Security & Separation

Allows access control without cluttering the Real Subject's business logic with security code.



Caching

Results of expensive operations can be cached inside the proxy, returning instantly on repeated calls.

Disadvantages



Response Latency

The extra layer of indirection can slightly delay the response, especially in Remote Proxies.



Boilerplate Code

You must duplicate the entire interface in the Proxy class, which can lead to code bloat for large interfaces.



Increased Complexity

Adding a proxy layer increases system complexity and can make debugging harder to trace.



Tight Coupling Risk

If the proxy is too tightly coupled to the real object, changes in the subject may break the proxy.

When to Use



Heavy Objects

We have resource-intensive objects that aren't always used. Use virtual proxy for lazy initialization.



Access Control

We need to restrict or monitor access to a service based on credentials or permissions.



Network Proxy

We are dealing with remote services or APIs that need a local stub.

When NOT to Use



Simple Logic

If the object is lightweight and access is internal/safe, don't add the overhead.



Direct Mutation

If the client needs to bypass control for direct state manipulation frequently.



High Speed Need

In real-time systems where even micro-seconds of indirection matter.

References

-  *Gang of Four (GoF)*: "Design Patterns: Elements of Reusable Object-Oriented Software" (Structural Catalog).
-  *Refactoring Guru (Alexander Shvets)*: Visual layout mappings of structural pattern implementations.
-  *Source Making*: Visual layout mappings of structural pattern implementations.
-  *IEEE Software Standards*: Best practices on structural composition and API decoupling metrics.

Questions & Answers

Feel Free to Ask Us

The End

Thank You

- Full Stack Alchemists Team